Pruning technikák

Állapottér bejárás és pruning

Számos kombinatorikai feladat megoldható visszalépéses kereséssel (backtracking), amely az összes lehetséges részmegoldást egy állapottér-fában járja be. Az ilyen keresések azonban gyakran exponenciális számú állapotot eredményeznek, ezért a teljes bejárás nem minden esetben végrehajtható hatékonyan. Ezen segít a **pruning** (visszametszés, vágás): célja, hogy ne vizsgáljunk meg olyan ágakat a keresési fában, amelyek biztosan nem befolyásolják a végeredményt.

Néhány példa:

Vannak számok: $3, 5, 8, 10$.

Válasszunk ki néhányat úgy, hogy az összeg ne haladja meg a 12-t.

Amikor

$3 + 5 + 8 = 16$

már tudjuk, hogy

$16 > 12$

Innen teljesen felesleges továbbmenni, ezért ezt az ágat levágjuk. Ez a pruning.

Egy másik példa: Az N királynő problémában soronként egy-egy királynőt helyezünk el. Már 3 királynőt elhelyeztünk a sakktáblán, és a negyediket próbáljuk lerakni a következő sorba. Ha egy mező egy már lerakott királynővel

- ugyanabban az oszlopban van,
- vagy ugyanazon az átlón van,

akkor azt az elhelyezést nem folytatjuk. Ez is pruning. A teljes sakktáblát nem kell végigvizsgálni.

Kapcsolata a backtrackinggel

A backtracking algoritmus általában így néz ki:

```
bejár(csúcs):
    ha kész megoldás
        kiír
    különben
        minden lehetőségre (ciklus)
            ha érvényes
                bejár(következő)
```

Pruninggal:

```
bejár(csúcs):
    ha biztosan rossz
        vissza
    ha kész megoldás
        kiír
    különben
        minden lehetőségre (ciklus)
            bejár(következő)
```

A különbség csupán egyetlen plusz ellenőrzés:
`ha biztosan rossz, akkor vissza`
Ez az egy sor rengeteg időt takaríthat meg.

Összefoglalva: A hatékonyság növelésének kulcstechnikája a **pruning** (visszametszés), amely során a keresés bizonyos ágait már korai fázisban elvetjük, ha azokból biztosan nem vezethetünk érvényes vagy optimális megoldáshoz. Ez lehet például egy feltétel sérülése, a célérték túllépése, vagy annak belátása, hogy a további bővítés már nem vezethet jobb eredményre.

Az első feladatban még nem használunk pruningot, a második változatában viszont igen, így jól látható a pruning jelentősége.

1. Feladat I. – A király őrtornyai

A Számok Királyságának határán N őrtorony áll. A király minden toronyba szeretne őrt állítani, azonban bizonyos tornyok egyszerre nem őrizhetők, mert jelzőtüzeik megzavarnák egymást.

Egy *őrbeosztás* a tornyok egy olyan részhalmaza, amelyben egyetlen tiltott toronypár sem szerepel egyszerre.

Feladatod az összes lehetséges őrbeosztás előállítása.

Bemenet

Az első sor két egész számot tartalmaz: N és M, ahol

- N az őrtornyok száma,
- M a tiltott toronypárok száma.

A következő M sor mindegyike két egész számot tartalmaz: a és b, ami azt jelenti, hogy az a és b sorszámú torony egyszerre nem őrizhető.

Korlátok

- $1 \leq N \leq 20$
- $0 \leq M \leq 100$

Kimenet

Első sor:

Az érvényes őrbeosztások száma.

Ezután minden őrbeosztást külön sorba kell kiírni.

A tornyok sorszámai minden sorban növekvő sorrendben szerepeljenek.

Az üres őrbeosztást (egy üres sort) elsőként kell kiírni. A nem üres őrbeosztásokat ezután lexikografikus sorrendben kell megjeleníteni.

Példa

Bemenet

```
4 2
1 2
2 3
```

Kimenet

```
10

1
1 3
1 3 4
1 4
2
2 4
3
3 4
4
```

Hint 1

Használj egy logikai mátrixot (`bool tiltott[MAX][MAX]`), amely tárolja, hogy mely tornyok nem lehetnek együtt! Az aktuális őrbeosztás tornyainak sorszámait egy `vector<int> aktualis`, a kész őrbeosztásokat egy `vector<vector<int» megoldasok` tárolja.

Hint 2

Írj egy függvényt (`lehet`), amely ellenőrzi, hogy az `idx` torony felvehető-e a jelenlegi részhalmazba! Ehhez az `aktualis` elemein kell végigmenned, és megnézned, hogy valamelyik `t` elemre igaz-e a `tiltott[idx][t]`.

Hint 3

A backtrack eljárásban minden toronyról döntünk, hogy bekerül-e. Két ág van: az `idx` tornyot vagy bevesszük (ha lehet), vagy kihagyjuk. Ha `idx > N`, akkor már minden toronyról döntöttünk, tehát az aktuális részhalmaz egy kész őrbeosztás.

Hint 4

Az 1. ágon: `if (lehet(idx))`, azaz megnézzük, hogy az `idx` kiválasztható-e. Ha igen, betesszük, meghívjuk a `backtrack(idx + 1)`-et, majd kivesszük. A 2. ágon az `idx` tornyot NEM választjuk ki: `backtrack(idx + 1)`.

Hint 5

Ez a bejárási sorrend nem lexikografikus (például az `1 3` előbb készül el, mint az `1`), ezért a végén rendezd a megoldásokat (`sort`)! Az üres őrbeosztás a rendezés után automatikusan az első lesz.

Megoldás

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int N, M;

const int MAX=21; //20 torony lehet max.
bool tiltott[MAX][MAX];

vector<int> aktualis;
vector<vector<int>> megoldasok;

/*
    Eldönti, hogy az idx torony hozzáadható-e a jelenlegi őrbeosztáshoz.
*/
bool lehet(int idx)
{
    for (int i = 0; i < (int)aktualis.size(); i++)
    {
        if (tiltott[idx][aktualis[i]])
        {
            return false;
        }
    }
    return true;
}

void backtrack(int idx)
{
    if (idx > N)
    {
        megoldasok.push_back(aktualis);
        return;
```

```
36      }
37
38      // 1. ág: bevesszük
39      if (lehet(idx))
40      {
41          aktualis.push_back(idx);
42          backtrack(idx + 1);
43          aktualis.pop_back();
44      }
45
46      // 2. ág: nem vesszük be
47      backtrack(idx + 1);
48  }
49
50  int main()
51  {
52      ios::sync_with_stdio(false);
53      cin.tie(nullptr);
54      cin >> N >> M;
55
56      for (int i = 0; i < M; i++)
57      {
58          int a, b;
59          cin >> a >> b;
60
61          tiltott[a][b] = true;
62          tiltott[b][a] = true;
63      }
64      backtrack(1);
65      // A bejárás sorrendje nem lexikografikus (pl. az [1, 3] előbb
66      // készül el, mint az [1]), ezért a végén rendezünk.
67      // Az üres vector a legkisebb, így az kerül előre.
68      sort(megoldasok.begin(), megoldasok.end());
69      cout << megoldasok.size() << "\n";
70      for (int i = 0; i < (int)megoldasok.size(); i++)
71      {
72          for (int j = 0; j < (int)megoldasok[i].size(); j++)
73          {
74              if (j > 0)
75              {
76                  cout << " ";
77              }
78
79              cout << megoldasok[i][j];
80          }
81
82          cout << "\n";
83      }
84      return 0;
```

```
85 }
```

2. Feladat II. – A király őrtornyai

A király átnézte az összes lehetséges őrbeosztást, és azt szeretné kiválasztani, amelyben a lehető legtöbb toronyban áll őr.

Feladatod meghatározni azt az őrbeosztást,

- amelyben egyetlen tiltott toronypár sem szerepel egyszerre;
- amely a lehető legtöbb tornyot tartalmaz.

Ha több maximális őrbeosztás is létezik, akkor a **lexikografikusan legkisebbet** kell kiírni.

Két őrbeosztás összehasonlításakor a kisebb első eltérő toronyszámot tartalmazó tekintendő lexikografikusan kisebbnek. Ha az egyik a másik prefixe (előtagja), akkor a rövidebb tekintendő kisebbnek.

Bemenet

A standard input első sorában két egész szám található: N és M, ahol

- N az őrtornyok száma,
- M a tiltott toronypárok száma.

A következő M sor mindegyike két egész számot tartalmaz: a és b, ami azt jelenti, hogy az a és b sorszámú torony egyszerre nem őrizhető.

Korlátok

- $1 \leq N \leq 25$
- $0 \leq M \leq 100$

Kimenet

Az első sorba írd ki a maximális őrbeosztás méretét!

A második sorba írd ki a kiválasztott tornyok sorszámait növekvő sorrendben.

Példa

Bemenet

```
6 5
1 2
1 4
2 3
3 6
5 6
```

Kimenet

```
3
1 3 5
```

Magyarázat

A példában három toronynál nagyobb érvényes őrbeosztás nem létezik. Háromelemű érvényes őrbeosztásból négy is van: $\{1, 3, 5\}$, $\{2, 4, 5\}$, $\{2, 4, 6\}$ és $\{3, 4, 5\}$. Ezek közül az $\{1, 3, 5\}$ a lexikografikusan legkisebb, ezért ezt kell kiírni.

Hint 1

Ismét három vector kellene: `forbidden` a tiltott párokra (minden toronyhoz a vele ütköző tornyok listája), valamint kettő az aktuális és a legjobb tornyokra: `current`, `best`.

Hint 2

Mielőtt kiválasztasz egy új tornyot, ellenőrizd, hogy nem ütközik-e valamelyik korábban kiválasztott toronnyal.

Hint 3

Alkalmazz **pruning** technikát!

Ha az aktuális részmegoldásban eddig x tornyot választottál ki, még legfeljebb r torony választható, és már ismert egy k méretű megoldás, akkor

$$x + r < k$$

esetén ezen az ágon már biztosan nem található jobb megoldás.

Hint 4

Ha

$$x + r = k,$$

akkor is elhagyható az ág, amennyiben a belőle előállítható legjobb megoldás sem lehet lexikografikusan kisebb a már megtaláltnál. (Ez egy további vágási lehetőség; a lenti megoldás csak a Hint 3-ban leírt vágást alkalmazza, egyenlőség esetén tovább keres.)

Hint 5

A lexikografikus összehasonlításhoz érdemes egy külön függvényt írni! Pl.

```
bool lexicographicallySmaller(const vector<int> &a,
                              const vector<int> &b) {
    return a < b;
}
```

A `vector`-ok `<` operátora éppen lexikografikusan hasonlít.

Megoldás

```cpp
#include <bits/stdc++.h>
using namespace std;

int N, M;

vector<vector<int>> forbidden;
vector<int> current, best;

/*
Ellenőrzi, hogy az új torony felvehető-e
a jelenlegi kiválasztottak mellé.
*/
bool ok(int x) {
    for (int y : current) {
        for (int z : forbidden[y]) {
            if (z == x) return false;
        }
    }
    return true;
}

/* Lexikografikus összehasonlítás: true, ha a < b */
bool lexicographicallySmaller(const vector<int> &a, const vector<int> &b)
 ↪  {
    return a < b;
}

/* Pruning: megnézzük, hogy van-e esély jobb megoldásra*/
bool canImprove(int idx, int currentSize) {
    int remaining = N - idx + 1;
    return currentSize + remaining >= (int)best.size();
}

void backtrack(int idx) {
    if (idx > N) {
        if (current.size() > best.size()) {
            best = current;
        }
        else if (current.size() == best.size()) {
            if (lexicographicallySmaller(current, best)) {
                best = current;
            }
        }
        return;
    }

    // PRUNING: ha innen már nem lehet jobb megoldás
```

```
    if (!canImprove(idx, current.size())) {
        return;
    }

    // 1. ág: NEM választjuk
    backtrack(idx + 1);

    // 2. ág: választjuk (ha lehet)
    if (ok(idx)) {
        current.push_back(idx);
        backtrack(idx + 1);
        current.pop_back();
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> N >> M;
    forbidden.assign(N + 1, vector<int>());

    for (int i = 0; i < M; i++) {
        int a, b;
        cin >> a >> b;
        forbidden[a].push_back(b);
        forbidden[b].push_back(a);
    }
    backtrack(1);
    cout << best.size() << "\n";

    for (int i = 0; i < (int)best.size(); i++) {
        cout << best[i];
        if (i + 1 < (int)best.size()) cout << " ";
    }
    cout << "\n";
    return 0;
}
```

3. Adott összegű részhalmazok – I.

Megjegyzés

A következő feladatban a pruning különösen fontos szerepet játszik, mivel a részhalmazok generálása során jelentősen csökkenti a bejárandó állapottér méretét.

Adott egy N darab pozitív egész számot tartalmazó tömb, valamint egy T célérték. A tömbben ismétlődő elemek is előfordulhatnak.

Feladatod, hogy hatékony visszalépéses kereséssel (backtracking) keresd meg az összes olyan részhalmazt, amelyek elemeinek összege pontosan T.

A megoldás során különös figyelmet kell fordítani a **pruning technikák** alkalmazására, mivel a naiv bejárás időben nem fér bele.

Fontos szabályok:

- Minden elemet legfeljebb egyszer használhatsz fel egy részhalmazban.
- A részhalmazok elemei mindig **növekvő (nem csökkenő) sorrendben** szerepeljenek. (Ismétlődő értékek miatt egy részhalmazban azonos értékek is lehetnek, például `1 1 6`.)
- Az eredményben a részhalmazokat **lexikografikus sorrendben** kell kiírni.
- Azonos részhalmaz nem szerepelhet kétszer, még ha az input tartalmaz ismétlődő értékeket is.

Bemenet

A standard input első sora egy N egész szám.

A második sor N darab pozitív egész számot tartalmaz.

A harmadik sor a célösszeget, T-t tartalmazza.

Korlátok:

- $1 \leq N \leq 100$
- $1 \leq A[i] \leq 50$
- $1 \leq T \leq 100$
- Garantált, hogy a megoldások száma legfeljebb 10^5.

Kimenet

Az összes megfelelő részhalmaz kiírása soronként, elemek szóközzel elválasztva.

Ha nincs megoldás, akkor:

```
Nincs megoldas
```

szöveget kell kiírni.

Példa

Bemenet

```
7
10 1 2 7 6 1 5
8
```

Kimenet

```
1 1 6
1 2 5
1 7
2 6
```

Megjegyzés a megoldáshoz

A feladat hatékony megoldásához backtracking szükséges, amelyet több szinten is pruning technikákkal kell optimalizálni:

- Ha az aktuális részösszeg meghaladja T-t, az ág nem folytatható.
- A bemenet rendezése után, ha egy elem nagyobb, mint a fennmaradó célösszeg, a további elemek vizsgálata elhagyható.
- Az azonos értékű elemeket egy rekurziós szinten belül ki kell hagyni a duplikációk elkerülésére.

Hint 1

A megoldáshoz szükség van 3 vector-ra!

```
vector<int> a;
vector<vector<int>> result;
vector<int> current;
```

A beolvasott értékek, az eredmények, és az aktuális részhalmaz elemeinek tárolására.

Hint 2

A hatékony megoldás kulcsa a rendezett tömb, a növekvő indexek mentén történő bejárás, valamint a korai visszavágás (pruning) alkalmazása minden olyan ágon, amely nem vezethet érvényes megoldáshoz.

Hint 3

Ha az összeg (`sum`) egyenlő T-vel, akkor megvan egy megoldás; ha nagyobb T-nél, akkor vágunk (pruning). Akkor is vágni kell (és a ciklusból kilépni), ha az `a[j] <= T - sum` feltétel hamis, mert a rendezés miatt a további elemek még nagyobbak.

Hint 4

Arra is figyelni kell, hogy ne legyen benne duplikáció! Egy rekurziós szinten belül egy adott értéket csak egyszer próbáljunk ki: jegyezzük meg az előzőleg kipróbált értéket (`prev`), és ha az aktuális elem ugyanekkora, hagyjuk ki.

Megoldás

```cpp
#include <bits/stdc++.h>
using namespace std;

int N, T;
vector<int> a;

vector<vector<int>> result;
vector<int> current;

/*
Backtracking kombinációs bejárás
i = aktuális index
sum = jelenlegi összeg
*/
void backtrack(int i, int sum) {

    if (sum == T) {
        result.push_back(current);
        return;
    }

    if (sum > T) return;

    if (i >= N) return;

    int prev = -1;

    for (int j = i; j < N; j++) {

        // duplikáció védelem egy szinten
        bool isDuplicate = false;
        if (j > i && a[j] == prev) {
            isDuplicate = true;
        }

        if (!isDuplicate) {

            // pruning: ha túl nagy, akkor innentől minden nagyobb is
            if (a[j] <= T - sum) {

                current.push_back(a[j]);
                backtrack(j + 1, sum + a[j]);
                current.pop_back();
            }
            else {
                // rendezett tömb miatt itt már nincs értelme továbbmenni
                break;
```

```
            }
            prev = a[j];
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> N;
    a.resize(N);
    for (int i = 0; i < N; i++) {
        cin >> a[i];
    }
    cin >> T;
    sort(a.begin(), a.end());
    backtrack(0, 0);
    if (result.empty()) {
        cout << "Nincs megoldas\n";
        return 0;
    }
    for (auto &v : result) {
        for (int i = 0; i < (int)v.size(); i++) {
            cout << v[i];
            if (i + 1 < (int)v.size()) cout << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

4. Adott összegű részhalmazok – II.

Adott egy N darab pozitív egész számot tartalmazó tömb, valamint egy T célérték. A tömbben előfordulhatnak ismétlődő elemek is. Feladatod, hogy megtaláld az összes olyan részhalmazt (az elemek sorrendje nem számít), amelyek elemeinek összege pontosan T-vel egyenlő.

Fontos szabályok:

- Minden elemet legfeljebb egyszer használhatsz fel egy részhalmazban.
- A talált részhalmazokat **növekvő (nem csökkenő)** sorrendben kell tárolni (azaz minden részhalmaz elemei növekvő sorrendben legyenek).
- A végeredményben a részhalmazok **lexikografikusan** (szótárrendszerűen) növekvő sorrendben jelenjenek meg.
- A megoldásod ne tartalmazzon duplikált részhalmazokat!

Bemenet

A standard input első sorában egy N egész szám található, amely a tömb elemeinek számát jelöli.

A második sorban N darab szóközzel elválasztott pozitív egész szám található (a tömb elemei).

A harmadik sorban egyetlen T egész szám található (a célösszeg).

Korlátok:

- $1 \leq N \leq 100$
- $1 \leq A[i] \leq 50$
- $1 \leq T \leq 100$
- Garantált, hogy a megoldások száma legfeljebb 10^5.

Kimenet

A standard kimenetre írd ki az összes olyan részhalmazt, amelyek összege T-vel egyenlő! Minden részhalmazt külön sorba írj, az elemeket szóközzel elválasztva!

Ha nincs egyetlen megfelelő részhalmaz sem, a kimenet egyetlen sort tartalmazzon:

```
Nincs megoldas
```

Példa

Példa bemenet

```
7
10 1 2 7 6 1 5
8
```

Példa kimenet

```
1 1 6
1 2 5
1 7
2 6
```

Magyarázat

A bemeneti tömb: $[10, 1, 2, 7, 6, 1, 5]$, a célösszeg $T = 8$.

A lehetséges részhalmazok, amelyek összege 8:

- $[1, 1, 6]$ (az első és a második 1-es, valamint a 6-os)
- $[1, 2, 5]$ (az egyik 1-es, a 2-es és az 5-ös)
- $[1, 7]$ (az egyik 1-es és a 7-es)
- $[2, 6]$ (a 2-es és a 6-os)

A [10] nem jöhet szóba, mert $10 > 8$. A részhalmazok lexikografikusan növekvő sorrendben jelennek meg. Nincs két egyforma részhalmaz, annak ellenére, hogy két darab 1-es is van a tömbben.

Hint 1

Használd a **backtracking** (visszalépéses keresés) algoritmust a részhalmazok előállításához!

Hint 2

Rendezd a tömböt növekvő sorrendbe!

Hint 3

Alkalmazz **metszést** (pruning) a hatékonyság növelésére:

- Ha az aktuális részösszeg már meghaladta T-t, ne menj tovább ezen az ágon!
- Ha az aktuális elem nagyobb, mint a maradék célösszeg, a ciklusból is kiléphetsz (rendezés miatt).
- Az azonos értékű elemeket egy szinten belül hagyd ki, ezzel elkerülve a duplikált részhalmazokat!

Hint 4

A duplikátumok kiszűréséhez egy rekurziós szinten belül tárold az előzőleg kipróbált értéket! A részhalmazok elemeit mindig növekvő sorrendben add hozzá!

Hint 5

A lexikografikus sorrend garantálható, ha a tömböt rendezve indítod, és a rekurzió során az elemeket növekvő index szerint vizsgálod.

Megoldás

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void backtrack(int start, vector<int>& candidates, int target,
               vector<int>& current, vector<vector<int>>& result) {
    // Ha elértük a célt, elmentjük a megoldást
    if (target == 0) {
        result.push_back(current);
        return;
    }
```

```
14
15      // Ha túlléptük, ezt az ágat nem vizsgáljuk tovább (METSZÉS 1)
16      if (target < 0) {
17          return;
18      }
19
20      int prev = -1;
21      for (int i = start; i < (int)candidates.size(); i++) {
22          // Duplikátumok kihagyása (METSZÉS 2) - continue nélkül
23          if (candidates[i] != prev) {
24              // Ha az aktuális elem már nagyobb, mint a maradék cél,
25              // a többi elem is nagyobb (rendezés miatt) -> kilépünk
                ↪  (METSZÉS 3)
26              if (candidates[i] > target) {
27                  break;
28              }
29
30              // Választás
31              current.push_back(candidates[i]);
32
33              // Rekurzív hívás: i+1, mert minden elemet csak egyszer
                ↪  használhatunk
34              backtrack(i + 1, candidates, target - candidates[i], current,
                ↪  result);
35
36              // Visszalépés
37              current.pop_back();
38
39              // Az aktuális érték beállítása az előzőnek a duplikátumok
                ↪  kihagyásához
40              prev = candidates[i];
41          }
42      }
43  }
44
45  int main() {
46      ios::sync_with_stdio(false);
47      cin.tie(nullptr);
48
49      int N;
50      cin >> N;
51      vector<int> candidates(N);
52      for (int i = 0; i < N; i++) {
53          cin >> candidates[i];
54      }
55
56      int T;
57      cin >> T;
58      // Rendezés: lehetővé teszi a metszést és a lexikografikus sorrendet
```

```
59      sort(candidates.begin(), candidates.end());
60      vector<int> current;
61      vector<vector<int>> result;
62
63      backtrack(0, candidates, T, current, result);
64
65      if (result.empty()) {
66          cout << "Nincs megoldas\n";
67      } else {
68          for (const auto& subset : result) {
69              for (int i = 0; i < (int)subset.size(); i++) {
70                  cout << subset[i];
71                  if (i < (int)subset.size() - 1) {
72                      cout << " ";
73                  }
74              }
75              cout << "\n";
76          }
77      }
78      return 0;
79  }
```

5. Őrzőkapuk

A Számok Királyságában minden lehetséges sorrend egy-egy útvonalat jelent a Királyi Kertben.

A kertbe azonban csak olyan sorozatok léphetnek be, amelyeket az Őrzőkapuk engedélyeznek. Minden kapu egy szabályt figyel: azt vizsgálja, hogy az éppen kialakuló sorrend „összhangban van-e” a királyság törvényeivel.

Ahogy haladsz előre és egyre több számot fűzöl össze, minden lépésnél újabb kapuhoz érsz. Ha a kapu úgy látja, hogy a jelenlegi részleges sorrend már biztosan nem folytatható érvényes úttá, akkor nem enged tovább — vissza kell fordulnod, és másik utat keresned.

A célod, hogy megtaláld az összes olyan teljes útvonalat, amely végig átjut az összes őrzőkapun, és eléri a kert végpontját anélkül, hogy bármelyik szabályt megsértené.

A siker kulcsa nem az, hogy minden utat kipróbálj, hanem hogy felismerd, mely utak vezetnek zsákutcába még azelőtt, hogy végigjárnád őket.

- Minden számot pontosan egyszer kell felhasználni.
- A permutációban nem állhat két azonos érték egymás mellett.
- A permutációban minden szomszédos párra igaz kell legyen, hogy az első elem kisebb vagy egyenlő a másodiknál VAGY a különbségük legalább K.

Formálisan, egy $p_1, p_2, \ldots, p_N$ permutáció akkor elfogadott, ha minden $1 \leq i < N$ esetén:

$$p_i \neq p_{i+1} \quad \text{és} \quad (p_i \leq p_{i+1} \quad \vee \quad |p_i - p_{i+1}| \geq K)$$

Fontos:

- Azonos értékű elemek esetén különböző indexű elemeknek számítanak (tehát külön felhasználhatók).
- A kimenetben a permutációkat lexikografikusan növekvő sorrendben kell kiírni.
- A megoldás során azonos permutáció nem szerepelhet többször.

Bemenet

Az első sorban N és K található.
A második sor N darab egész számot tartalmaz.
Korlátok:

- $1 \leq N \leq 10$
- $1 \leq K \leq 100$
- $1 \leq A[i] \leq 100$
- Garantált, hogy a megfelelő permutációk száma legfeljebb 10^5.

Kimenet

Írd ki az összes megfelelő permutációt soronként, szóközökkel elválasztva.
Ha nincs megoldás, akkor az alábbi szöveget:

```
Nincs megoldas
```

Példa

Bemenet

```
4 3
1 3 5 2
```

Kimenet

```
1 2 3 5
1 3 5 2
1 5 2 3
2 3 5 1
2 5 1 3
3 5 1 2
5 1 2 3
```

Magyarázat

A rendezett bemenet: $1, 2, 3, 5$, és $K = 3$. Egy szomszédos pár akkor megengedett, ha növekvő, vagy ha a csökkenés legalább 3. A 3-as szám után tehát csak nagyobb szám állhat (a csökkenés legfeljebb 2 lenne), az 5-ös után viszont az 1 és a 2 is következhet. Ezért például a `2 5 3 1` nem megfelelő, mert az $5 \to 3$ csökkenés csak 2.

Megjegyzés

A feladatot visszalépéses kereséssel (backtracking) oldjuk meg.

A naiv megoldás minden permutációt előállítana, és csak a végén ellenőrizné ($O(N \cdot N!)$, $N = 10$ esetén $10! = 3\,628\,800$ permutáció), ezért érdemes a **pruning alkalmazása**: a hibás részleges permutációkat azonnal elvetjük.

Hint 1

A megoldás során építsd fel a permutációt lépésről lépésre, és minden lépésnél ellenőrizd a feltételeket.

Hint 2

Ha egy részleges permutáció már sérti a szomszédsági feltételt, az ágat azonnal el lehet vetni (pruning).

Hint 3

A bemenet rendezése segíthet a lexikografikus sorrend biztosításában.

Hint 4

Az azonos értékű elemek kezelésére figyelj: bár külön indexen szerepelnek, ugyanazt a permutációt adnák többször. Ezért egy rekurziós szinten belül egy adott értéket csak egyszer próbáljunk ki (jegyezzük meg az ezen a szinten utoljára kipróbált értéket). Így nem kell a kész permutációk között utólag keresni a duplikátumokat.

Megoldás

```cpp
#include <bits/stdc++.h>
using namespace std;

int N, K;
vector<int> a;

vector<bool> used;
vector<int> perm;
vector<vector<int>> result;

bool ok(int last, int cur) {
    if (last == -1) return true;
    if (last == cur) return false;
    if (last <= cur) return true;
    if (abs(last - cur) >= K) return true;
    return false;
}

void backtrack() {
```

```
20      if ((int)perm.size() == N) {
21          result.push_back(perm);
22          return;
23      }
24
25      int last = perm.empty() ? -1 : perm.back();
26
27      int prevUsedValue = INT_MIN;
28      bool hasPrev = false;
29
30      for (int i = 0; i < N; i++) {
31
32          if (!used[i]) {
33
34              // duplikáció pruning (azonos értékek egy szinten)
35              bool duplicateSkip = false;
36              if (hasPrev && a[i] == prevUsedValue) {
37                  duplicateSkip = true;
38              }
39              if (!duplicateSkip) {
40
41                  // szomszédsági feltétel pruning
42                  if (ok(last, a[i])) {
43
44                      used[i] = true;
45                      perm.push_back(a[i]);
46
47                      backtrack(); //itt a rekurzió
48
49                      perm.pop_back(); //"visszatörlés"
50                      used[i] = false;
51
52                      prevUsedValue = a[i];
53                      hasPrev = true;
54                  }
55              }
56          }
57      }
58  }
59
60  int main() {
61      ios::sync_with_stdio(false);
62      cin.tie(nullptr);
63      cin >> N >> K;
64      a.resize(N);
65      used.assign(N, false);
66      for (int i = 0; i < N; i++) {
67          cin >> a[i];
68      }
```

```
69
70      sort(a.begin(), a.end());
71      backtrack();
72      if (result.empty()) {
73          cout << "Nincs megoldas\n";
74          return 0;
75      }
76      for (auto &v : result) {
77          for (int i = 0; i < (int)v.size(); i++) {
78              cout << v[i] << (i + 1 < (int)v.size() ? ' ' : '\n');
79          }
80      }
81      return 0;
82  }
```